Introduction



StockShark is a web application designed to provide users with quick and accessible stock data, powered by real time information fetched from the third-party Finance API. This project aims to deliver a simple tool that enables users to compare historical stock data and make informed decisions.

Our Aim:

Plan & Develop an architecture that will efficiently manage the operations of this application.

Tools Used:

IJ

Apache SOFTWARE FOUNDATION

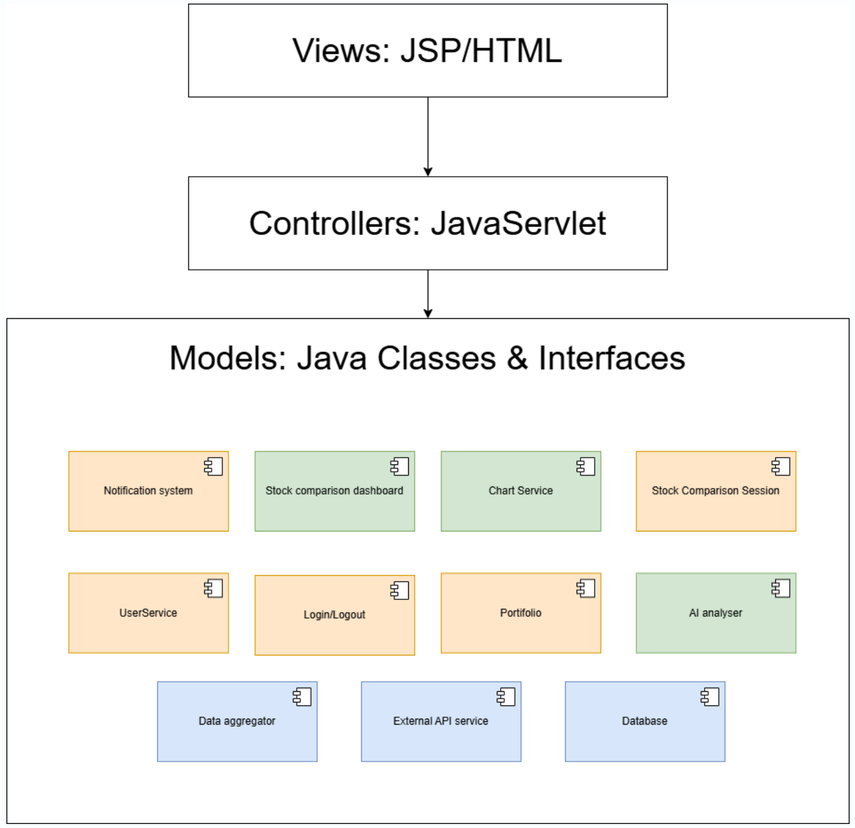
JAKARTA EE

HTML

Java

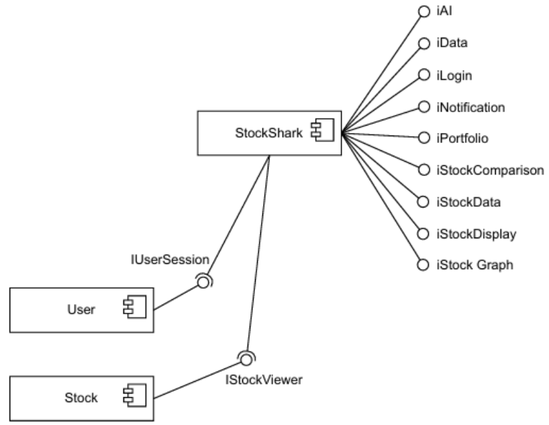
MySQL

The Architecture

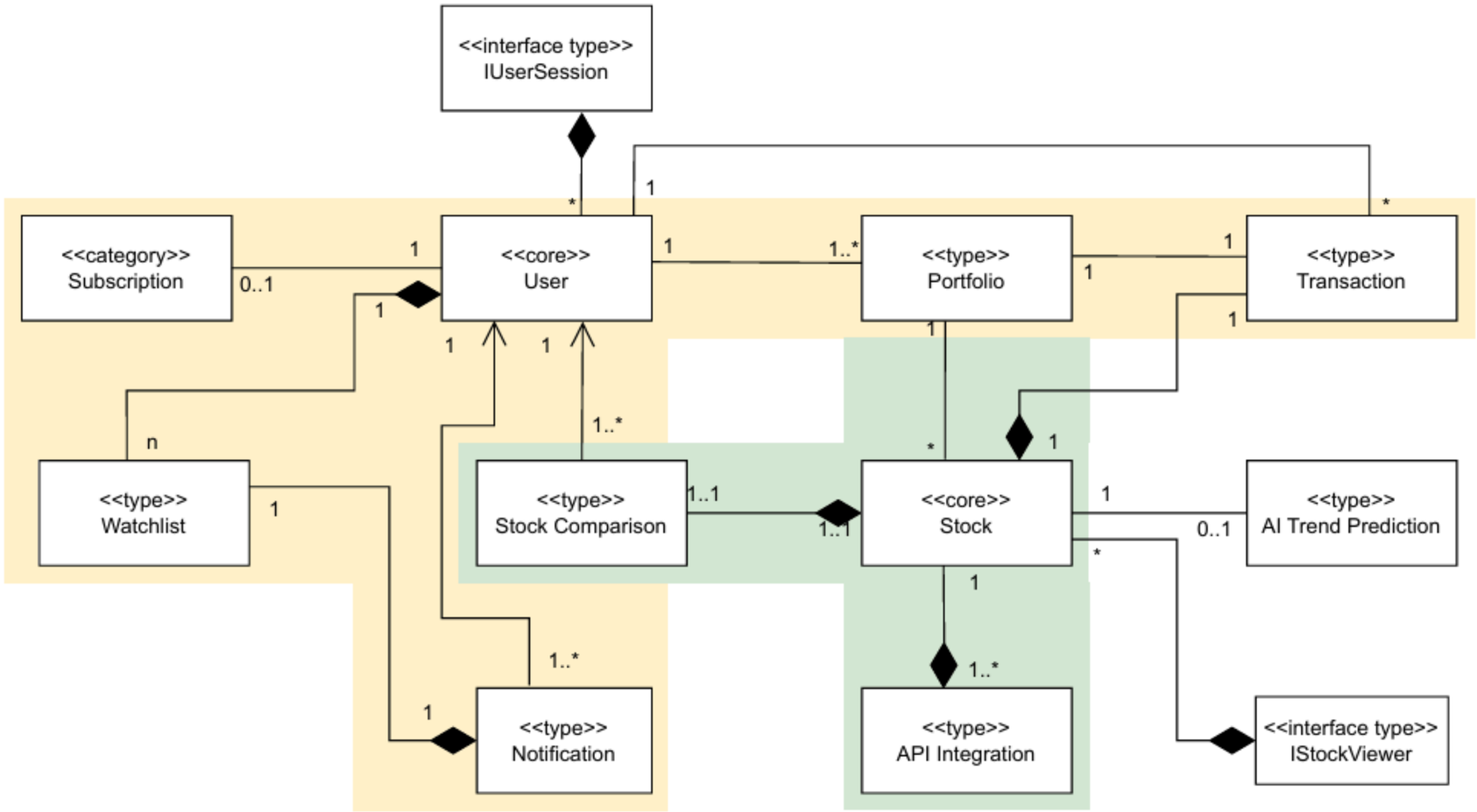


The architecture is divided into :

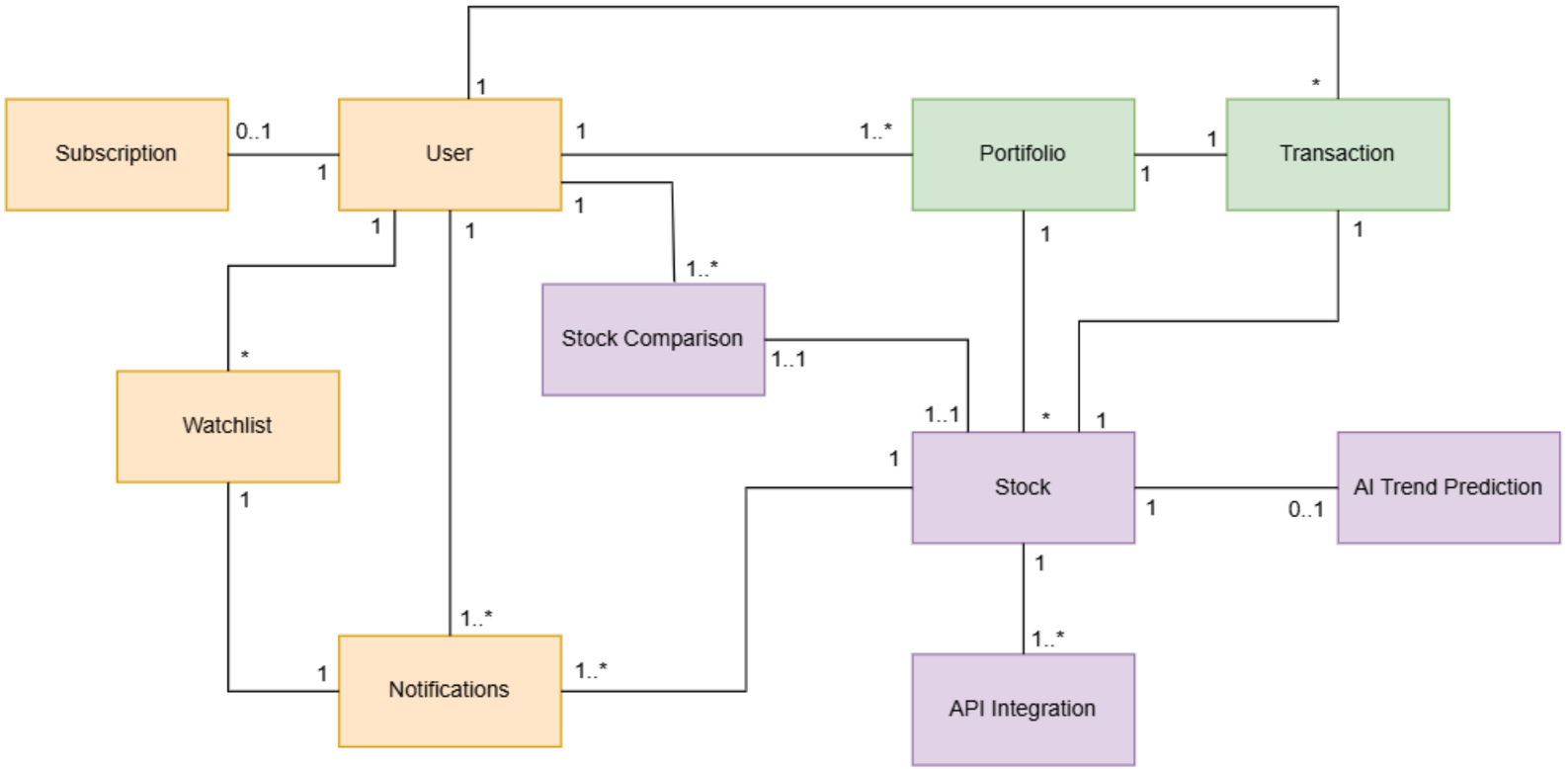
- Observer pattern:
  - The notification system listens for changes made in the database which periodically collects the latest updates from the Stocks API.
  - Then the other side responsible for the port - I notifications publishes the information to the user service
- Model view controller: this is employed on a much broader scope than the individual component connections. In order for the user to interact with the system. We have developed the UI using Java Service Package files and HTML.
  - Java itself is not a web oriented language. Therefore .jsp files are necessary inorder to inject our functions within html.
  - The controllers are responsible for managing the communication between the two.
- 5tiered architecture
  - This means our architecture also satisfies the five tiered architecture. Client side interface, business logic, and database server which is provided using Xammp.
  - Sub components employ 2 tiered architecture
  - Where We are required to source the stock data from Alpha vantage (our External source). We have developed a database client that can request data from this API.



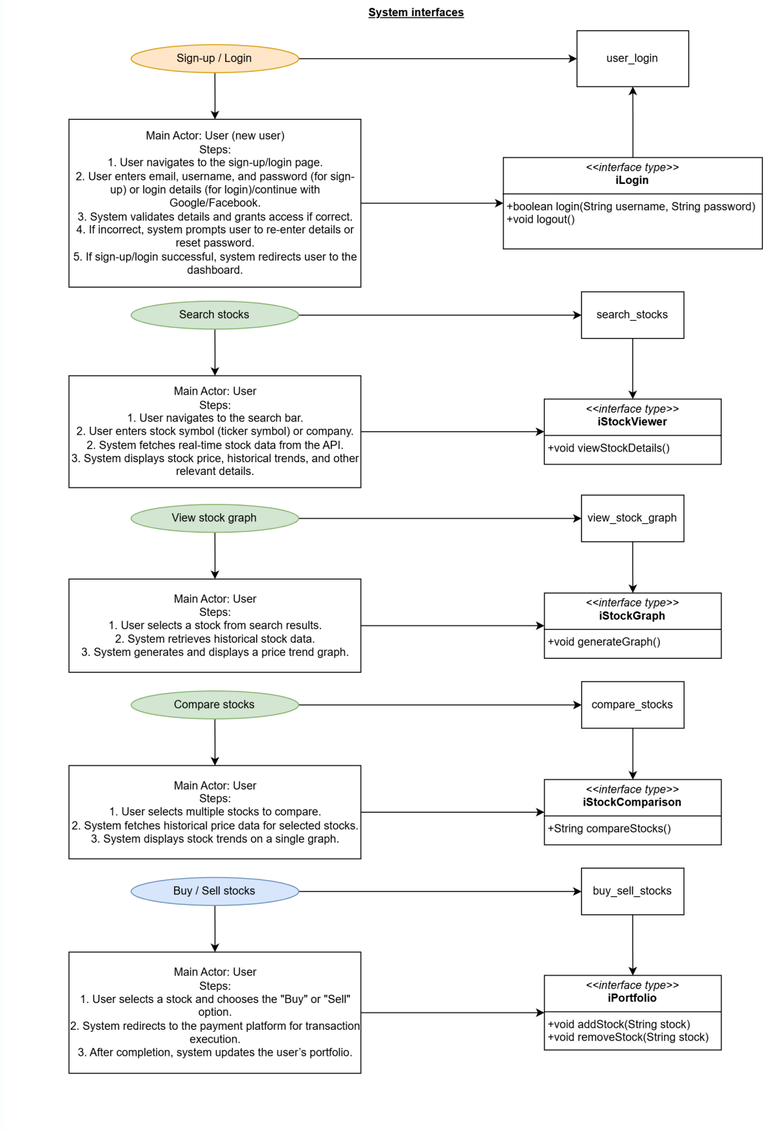
StockShark Components

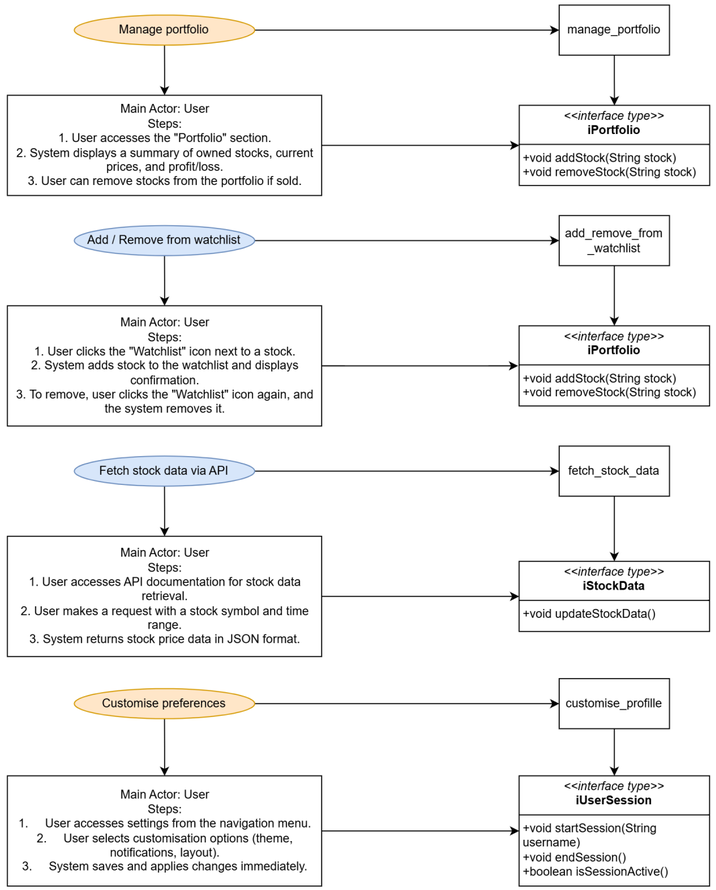


Business Concept Model



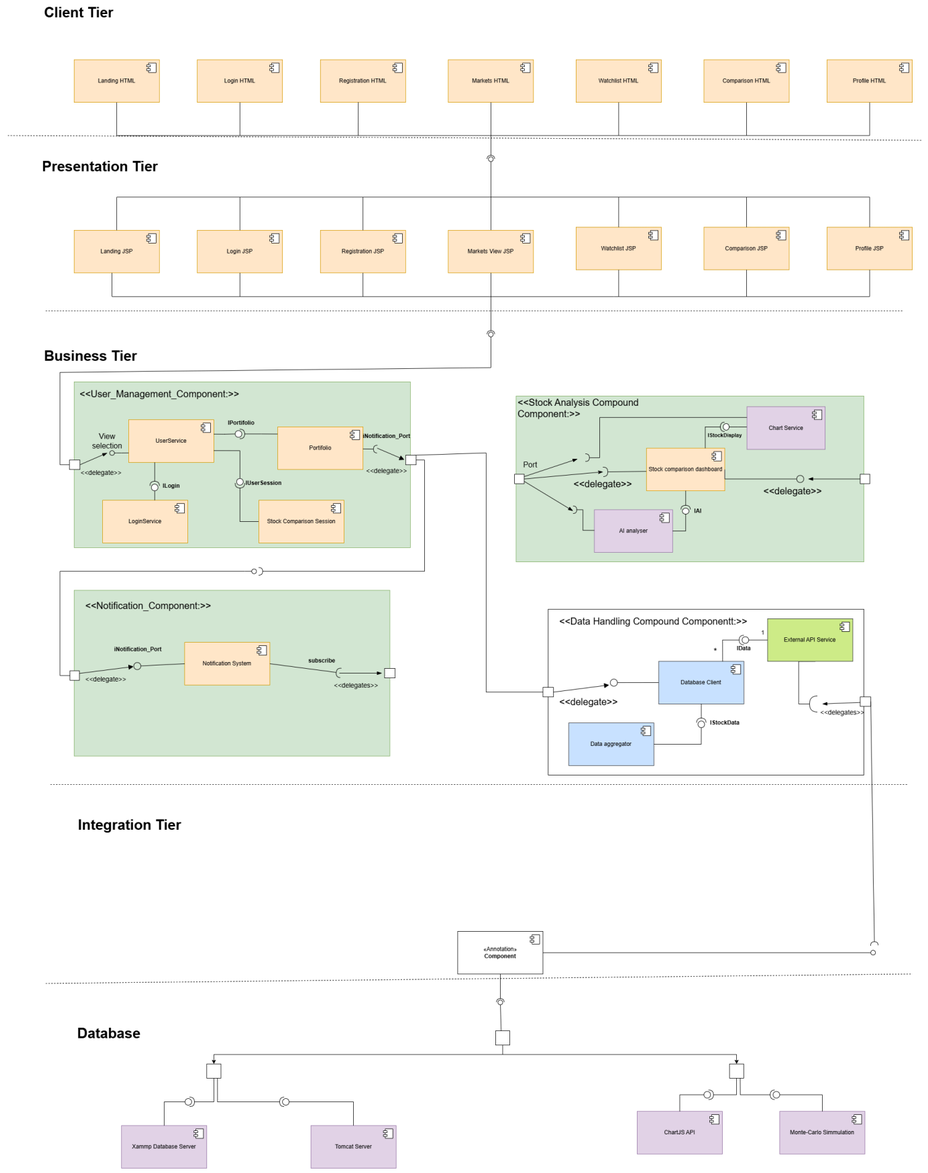
System Interfaces





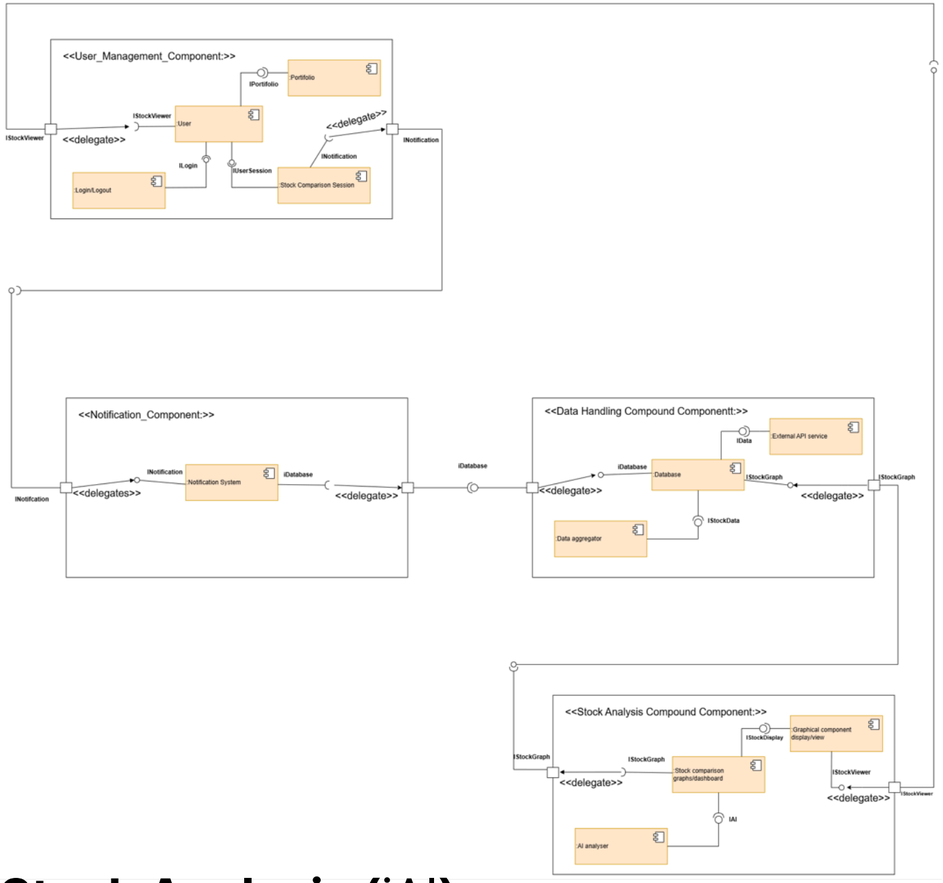
Compound Components

A Detailed look into the architecture and its components



CORE COMPONENTS

STOCKSHARK CONSISTS OF KEY COMPONENTS THAT WORK TOGETHER TO DELIVER STOCK MARKET ANALYSIS AND PORTFOLIO MANAGEMENT.



01 User Management (IUserSession, iLogin)

The ‘User’ component acts as the central hub, managing user sessions (IUserSession), interactions, and access to the application’s features.
This component is responsible for managing user interactions, handling login and logout operations through the ‘iLogin’ interface, and granting access to essential system features. User can manage their portfolios, compare stocks, and receive notifications, making this a critical part of the application

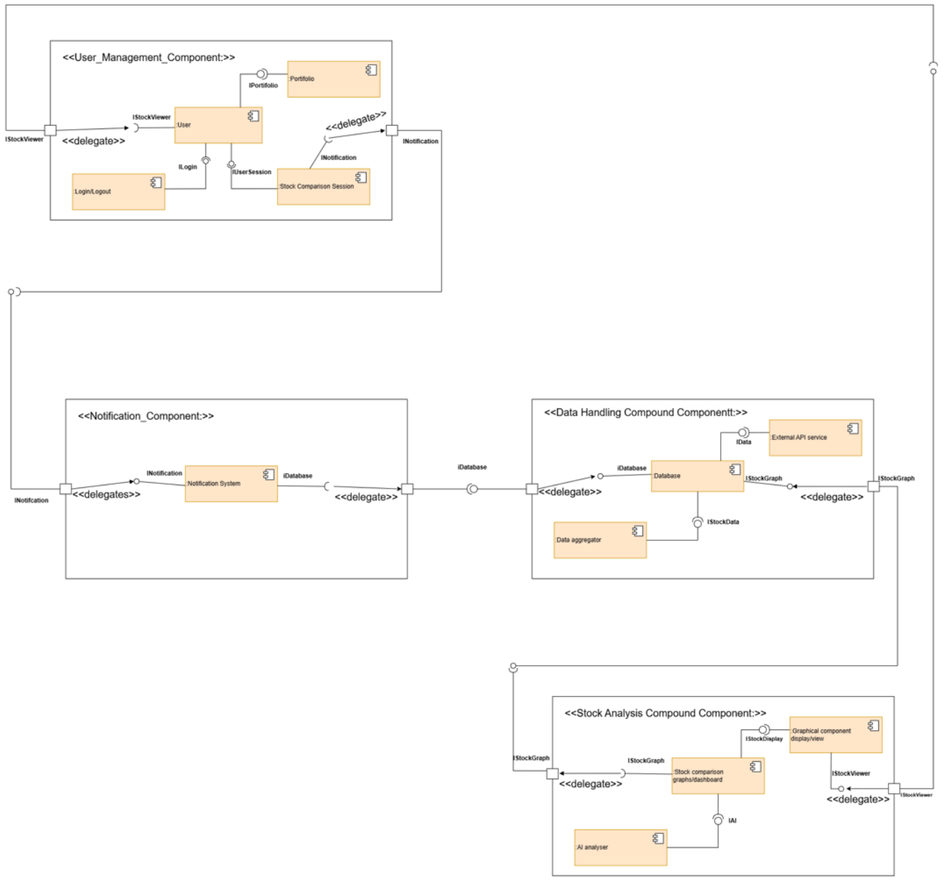
02 Stock Data Handling (iStockData, Database)

The ‘Stock’ component represents individual stock information, crucial for both portfolio management and analysis.
This class is used by the portfolio to store the stock information, and by the API Integration to get the stock data. Additionally, the ‘DataAggregator’ abstract class, which implements ‘iStockData’, ensures stock information is continuously fetched and updated.

03 Stock Analysis (iAI)

The system incorporates AI-powered stock analysis through the ‘AiAnalyser’ class, which implements the ‘iAI’ interface. This component is designed to analyze stock trends and predict potential market movements.

CORE COMPONENTS



04 Graphical Visualization (iStockDisplay)

StockShark includes a visualization module managed by the ‘GraphicalComponent’ class, which implements ‘iStockDisplay’. This component integrates with JavaFX ‘LineChart’ to display stock trends graphically. The ‘GraphicalComponent’ relies on ‘iStockGraph’ to generate stock performance graphs, allowing users to interact with stock data in an intuitive and visually engaging manner.

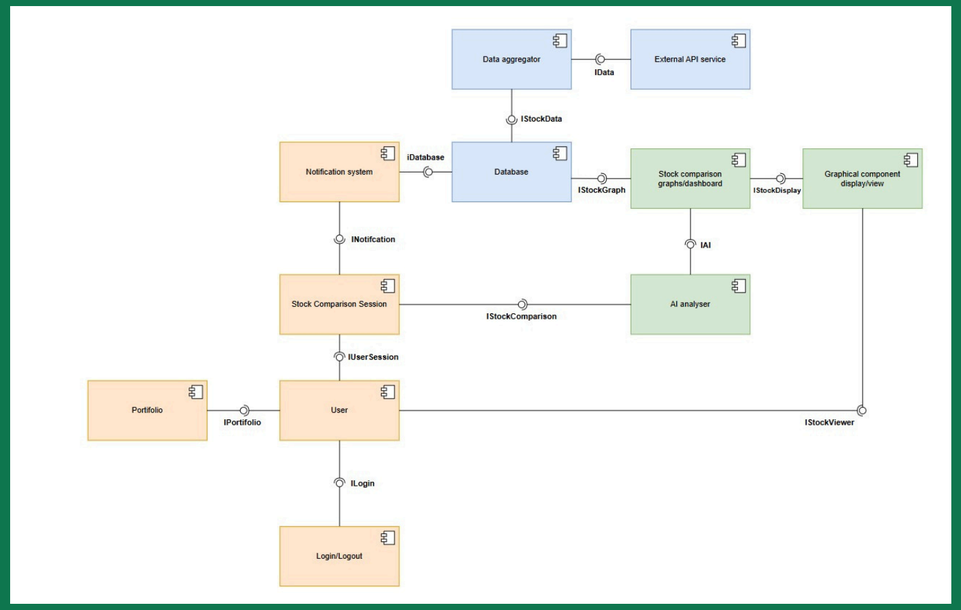
05 Stock Comparison (iStockComparison)

The ‘StockComparisonSession’ class, implementing ‘iStockComparison’, enables users to compare different stocks effectively. Additionally, the ‘NotificationSystem’ class, which implements ‘iNotification’, ensures users stay informed by sending real-time alerts on stock movements, portfolios changes, and market updates. These features enhance user engagement and help investors make timely decisions.

06 Notification (iNotification)

Notification sends real-time alerts on stock market changes and keeps the users updated on portfolio movements and significant events.

COMPOUND COMPONENTS



The StockShark system follows a modular and layered architecture, ensuring scalability, maintainability, and efficiency. Below is an overview of the key architectural components, their interfaces, and their justification for use.

Observer Pattern (Notification System):

The ‘NotificationSystem’ listens for stock updates and sends real-times alerts. Helps keep users informed on stock movements and portfolio changes.

MVC (Model-View-Controller):

The ‘GraphicalComponent’ acts as the Vies displaying stock trends. The ‘StockComparisonSession’ and ‘AIAnalyser’ acts as the Controller, handling user input and processing stock data. The ‘stock’, ‘Portfolio’, and ‘Database’ serve as the Model managing data.

Facade Pattern (User Component):

The ‘User’ component acts as a central hub, providing a unified interface to manage user sessions, portfolios, and stock comparisons. Hides the complexity of multiple interconnected components.

Factory Pattern (Data Aggregation & API Integration):

the ‘DataAggregator’ implements ‘iStockData’, ensuring stock data is fetched consistently from different sources.

HOW GIVEN ARCHITECTURE PROVIDES A SOLUTION TO USER STORIES

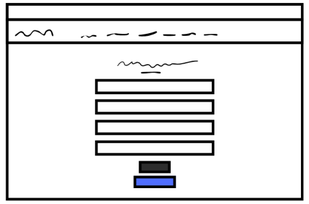
- **User wants to manage their portfolio** → The ‘User’ component facilitates stock additions/removals, storing data via the ‘Database’
- **User wants AI-Driven stock predictions** → The ‘AIAnalyser’ component processes stock data and provides trend forecasts.
- **User wants real-time stock updates** → The ‘NotificationSystem’ alerts users of important stock movements.
- **User wants a visual representation of stock trends** → The ‘GraphicalComponent’ integrates with JavaFX ‘LineChart’ to display stock trends.



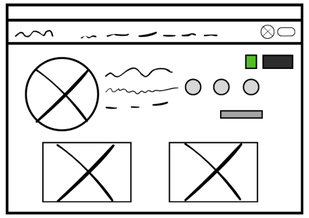
REUSEABLE UI COMPONENTS

Low-fidelity Wireframes

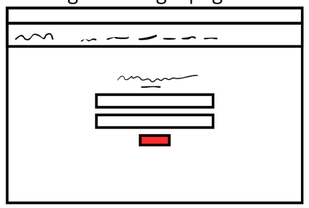
Create account page



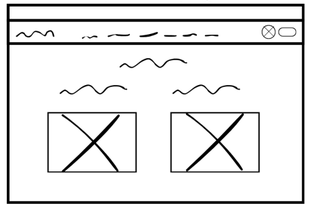
Profile page



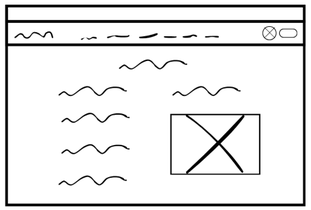
Existing users login page



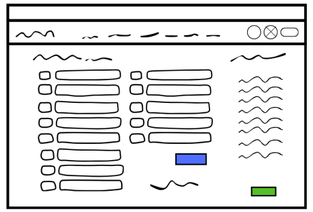
Stock comparison page



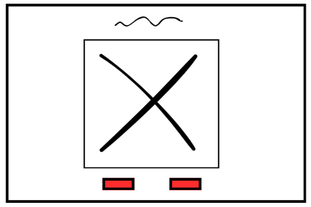
Watchlist



Stocklist



Landing page



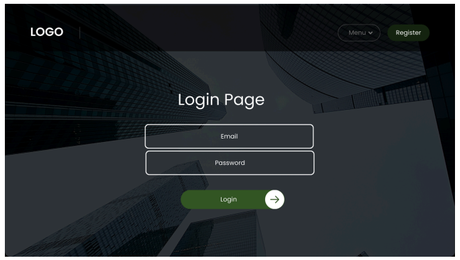
Why use reusable UI components?

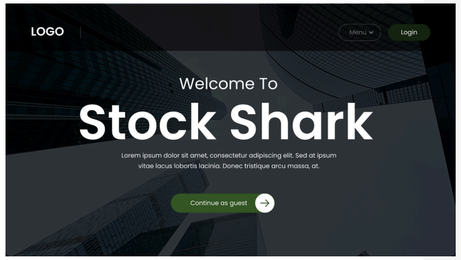
By using reusable components in our project, we saved time, reduced redundancy, and made the system easier to maintain and scale.

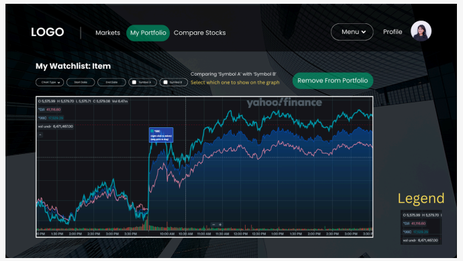
What are some examples in our system?

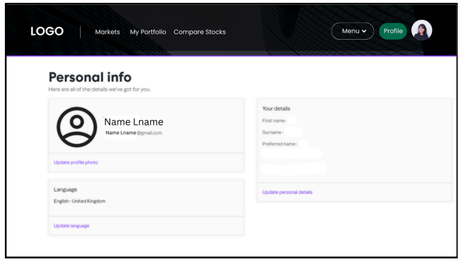
Some examples include our login/account creation forms with user input fields and graphs that we generated with the help of Chart.js tools.

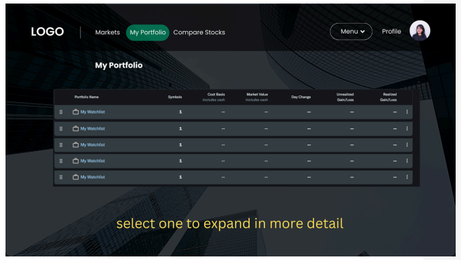
High-fidelity Wireframes

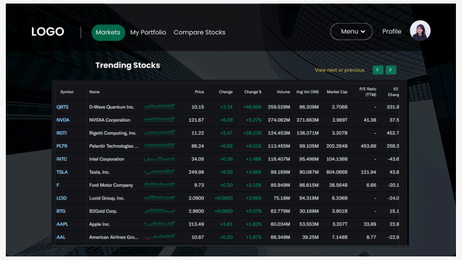












SOA

SERVICE ORIENTATED ARCHITECTURE



User → Login → Session Created → Data Requested → Graph Displayed

01 What are they?

Service-Oriented Architecture (SOA) is an architectural design principle where the system is divided into a set of independent services that communicate over a network. Each service handles a specific task and can be reused by multiple clients.

02 How did we implement these principles?

- User Authentication: Handles user login and session management.
- Stock Data Retrieval: Fetches stock data from external APIs.
- Data Display: Processes and presents the stock data on user-facing graphs.

03 How did this benefit our system?

SOA enhances modularity, scalability, and interoperability, allowing independent service updates and easy integration. It also improves maintainability by enabling isolated changes without disrupting the entire system.

MOST DOMINANT STYLES USED

01 Adapter

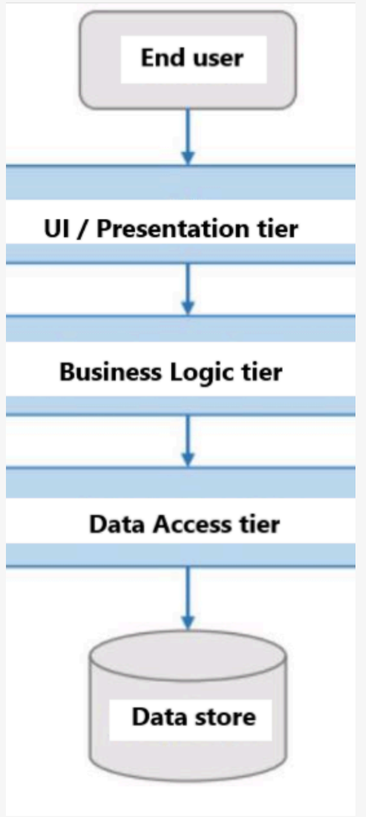
The Adapter pattern in our project is used to integrate external stock APIs. It converts the raw API data into a format that our system can easily process, allowing us to display stock prices and perform analysis without tightly coupling the system to the API.

02 Model View Controller

- Model: Contains the application's data and logic (e.g., UserService and LoginService).
- View: The JSP pages (like profile.jsp, register.jsp) are responsible for displaying the user interface.
- Controller: The Servlets (e.g., RegisterServlet, LoginServlet) act as the controller, processing user input, invoking the appropriate business logic, and selecting the view to render.

03 N-Tiered Architecture (5-Tier)

- Presentation Layer: The JSP files (e.g., profile.jsp, register.jsp) represent the user interface where users interact with the system.
- Business Logic Layer: The services (like UserService, LoginService) handle the core logic of the application, such as managing user registration, profile, and authentication.
- Data Access Layer: The data access layer handles interactions with the database, via SQL, to store and retrieve user data.



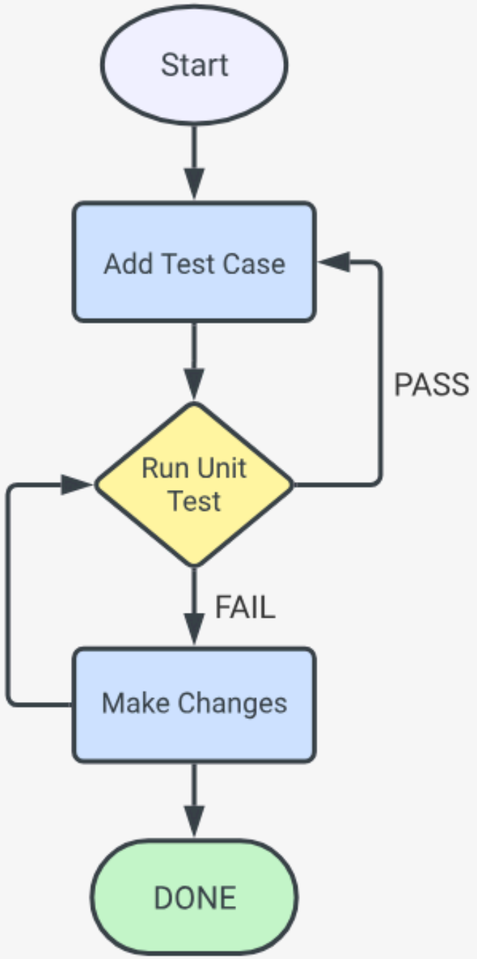
UNIT TESTING

01 What is Unit Testing?

Testing individual components (functions/methods) in isolation to ensure they behave as expected.

02 What did we do?

- Written Functions: Each function is a "unit" that can be tested.
- SQL for Testing Data: Used SQL queries to test data directly, bypassing the API with limited runs.
- GitHub Pull Requests: We worked on separate parts of the project and used pull requests for collaboration, code review, and merging.
- Manual Testing: Initially tested manually to verify functionality before committing changes.



03 Benefits we gained from this:

- Catch Bugs Early: Identify issues early without affecting the API.
- Refactor Confidently: Safely refactor code, knowing unit tests will ensure functionality remains intact.
- Collaboration and Code Review: Pull requests allowed for efficient collaboration and peer reviews.
- Automated Checks: SQL-based tests and unit tests ensure data integrity without hitting the API.